Prvi praktični test iz Algoritama i struktura podataka

Prvi praktični test održan 25.11.2023.

Ime i prezime: _____

Broj indeksa: _____

Instrukcije za rešavanje testa

1. Vreme izrade testa je **90 minuta**.
2. Ovo je *closed book test*, nije dozvoljeno korišćenje dodatnih materijala.
3. Pre početka testa:
    a. Vodite računa da imate sve spremno što vam je neophodno.
    b. Isključite mobilne telefone i odložite ih na katedru.
4. Ukupan broj bodova na testu je 25 bodova.
5. Na lokalnom disku se nalazi šablon koji je potrebno preuzeti i koristiti prilikom rešavanja zadataka. Detaljne instrukcije će biti date od strane dežurnog asistenta.
6. U pruženom šablonu obavezno upisati broj indeksa na nazbačenom mestu.
7. LINQPad fajl koji se koristi preimenovati prema sledećem šablonu: **AISP2024_Test1_GrupaG_IT1g2023**
8. Ukoliko algoritam (rešenje zadatka) ima neku sintaksnu ili logičku grešku što uslovljava nemogućnost pokretanja i testiranja programa, najveći broj bodova koji se može osvojiti za taj algoritam je **50% od ukupnog broja bodova.**

Pre početka testa napišite ime, prezime i broj indeksa.

PREPISIVANJE TOKOM TESTA NIJE MORALNO I ETIČKI PRIHVATLJIVO!

Studenti koji budu prepisivali mogu biti udaljeni sa testa. Za studente koji budu prepisivali test će se smatrati nevažećim!

Ovim putem se izjašnjavam da sam upoznat/a sa pravilima ponašanja na testu kao i sa posledicama u slučaju kršenja istih.

Potpis studenta:_____

Zadatak 1 – *Binarno pretraživanje*

Napisati ***sort*** metodu klase ***BinarySearch*** .

Dodatne napomene za rešavanje zadatka:

- Ulaz je sortiran niz celih brojeva
- Izlaz je true ili false, u zavisnosti da li je broj pronađen
- U template-u se nalazi klasa ***SortedArrayGenerator*** koju treba iskoristiti za generisanje sortiranog niza slučajnih brojeva
- Kreirati klasu ***BinarySearch*** i u okviru nje generisati metodu ***public static bool search(long[] arr, int size, long value)***
- Testirati u okviru ***Main*** metode

Zadatak 2 – *Promena redosleda n elemenata sa kraja reda, koristeći stek*

Implementirati funkciju koja za zadati **red** celih brojeva obrće redosled samo poslednjih n elemenata reda, koristeći pomoćni **stek**. Preostali elementi reda ostaju u istom redosledu.

Specifikacija funkcije:

- Funkcija prima broj n, gde je n broj elemenata sa kraja reda koje treba obrnuti.
- Funkcija treba koristiti stek (***stack***) kao pomoćnu strukturu.
- Elementi pre poslednjih n treba da ostanu u originalnom redosledu.

Dodatni zahtevi za rešavanje zadatka:

- Algoritam pisati u okviru klase ***Queue.***
- Zahtevan potpis metode: ***public void ReverseLastNQueueElements (int n)***

Primer1:

- Originalni red: 1 2 3 4 5 6 7 , n = 4
- Red nakon obrtanja poslednjih 4 elementa: 1 2 3 7 6 5 4

Primer2:

- Originalni red: 5 3 2 1 8 , n = 5
- Red nakon obrtanja poslednjih 5 elementa: 8 1 2 3 5

Primer3:

- Originalni red: 1 2 3 4 5 6 , n = 7
- Nevažeća vrednost za n.

Zadatak 3 – *Najduži segment sa neparnim i parnim čvorovima*

Čvorovi u povezanoj listi mogu formirati **segment sa naizmenično neparnim i parnim vrednostima** ako zadovoljavaju sledeće uslove:

1. Svaki čvor u segmentu ima vrednost koja je različita po parnosti od vrednosti prethodnog čvora.
2. Segment mora imati najmanje 3 čvora.

Zadatak je pronaći i vratiti:

1. **Dužinu najdužeg segmenta sa naizmenično neparnim i parnim čvorovima.**
2. **Početnu vrednost segmenta.**
3. **Krajnju vrednost segmenta.**

Ako nijedan takav segment ne postoji, treba vratiti niz [-1, -1, -1].

Dodatni zahtevi:

- Rešavaj ovaj zadatak pomoću strukture liste (*LinkedList*) iz šablona zadatka.
- Implementiraj ga u okviru metode: ***public int[] findLongestSegment()***

<u>Primer1:</u>

- Input 1: lista: 1 -> 3 -> 5 -> 7
- Output 1: [-1, -1, -1]

Objašnjenje: Nema naizmeničnih vrednosti po parnosti, pa ne postoji validan segment.

Primer2:

- Input 2: lista: 2 -> 3 -> 4 -> 5 -> 6 -> 7
- Output 2: [6, 2, 7]

Objašnjenje: Najduži segment je 2 -> 3 -> 4 -> 5 -> 6 -> 7, sa dužinom 6. Počinje na vrednosti 2 i završava se na vrednosti 7.